You have ₹15 and go to a shop where each chocolate costs ₹1. The shopkeeper also offers a deal: you can exchange 3 empty chocolate wrappers for 1 additional chocolate. So, how many chocolates can you eat at maximum?

Solution: With ₹15, you buy and eat **15 chocolates** - you now have **15 wrappers**.

- Exchange **15 wrappers** for **5 more chocolates** (since 3 wrappers = 1 chocolate) - Now you have **5 more wrappers**.
- Exchange **3 wrappers** for **1 more chocolate-** now you have **2 + 1 = 3 wrappers** (2 remaining + 1 from the new chocolate).
- Exchange these **3 wrappers** for **1 more chocolate**.

Total chocolates eaten = 15 + 5 + 1 + 1 = 22 chocolates.

A man fell into a **50m** deep well. He climbs **4 meters** up and slips **3 meters** down in one day. How many days would it take for him to come out of the well?

Solution:

- On the first day, the man climbs **4 meters** up and slips **3 meters** down; therefore, he only climbs **1 meter** up total.
- On the second day, again he climbs **1 meter** up, so the total distance climbed is **2 meters** till the second day. Therefore, the man climbs 1 meter every day.

Now, as per the above pattern, on the **46th day**, he must have climbed 46 meters. So on the **47th day**, he climbs full (46 + 4) 50 meters, and after

that, he will not slip as he is already out of the well, so the answer is **47 days**.

A factory has **N machines**. Each machine produces products at a fixed rate.

The `i-th` machine takes exactly `t[i]` seconds to produce **one product**.

All machines can work **simultaneously**, and each machine can continue producing products without any limit.

Given the production time of every machine and a target number `K`, determine the **minimum amount of time required to produce at least `K` products**.

Input

The input consists of:

- The first line contains two integers `N` and `K`:
  - `N` — the number of machines.
  - `K` — the minimum number of products that must be produced.
- The second line contains `N` integers:

t[1], t[2], ..., t[N]

where `t[i]` represents the number of seconds required by machine `i` to produce one product.

Constraints

1 ≤ N ≤ 2 × 10^5
1 ≤ K ≤ 10^18
1 ≤ t[i] ≤ 10^9

Output

Print a single integer:

the minimum number of seconds required
to produce at least K products

Example

Input

3 10

2 3 7

Output

12

Input
5 1
10 7 3 20 5

Output
3

Input
5 50
1 100 100 100 100

Output
50

Solution:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

using ll = long long;

// Can we produce at least K products in 'time' seconds?
bool canProduce(vector<ll>& t, ll K, ll time) {
    ll products = 0;

    for (ll x : t) {
        products += time / x;

        // Avoid unnecessary overflow / large calculations
        if (products >= K)
            return true;
    }

    return false;
}

int main() {
    int N;
    ll K;

    cin >> N >> K;
```

```
    vector<ll> t(N);

    for (int i = 0; i < N; i++) {
        cin >> t[i];
    }

    // Minimum possible time
    ll low = 0;

    // Maximum possible answer:
    // Fastest machine alone produces all K products
    ll fastest = *min_element(t.begin(), t.end());
    ll high = fastest * K;

    ll answer = high;

    while (low <= high) {
        ll mid = low + (high - low) / 2;

        if (canProduce(t, K, mid)) {
            // We can produce K products.
            // Try to find a smaller time.
            answer = mid;
            high = mid - 1;
        }
        else {
            // Not enough products.
            // Need more time.
            low = mid + 1;
        }
    }

    cout << answer << endl;

    return 0;
}
```

****

You are given the positions of n houses located along a straight road. You need to install **at most k Wi-Fi routers** so that every house is covered by at least one router.

Each router has the same **integer radius R**.

A router installed at position x can cover every house whose position lies within the range:

[x - R, x + R]

Your task is to find the **minimum possible integer value of R** such that all houses can be covered using **at most k routers**.

Important

- The house positions are integers.
- The radius R must be a **non-negative integer**.
- You may place a router at any position on the road.
- A router can cover multiple houses.
- Return the **minimum integer radius** required.

Input

- The first line contains an integer n, the number of houses.
- The second line contains n integers representing the positions of the houses.
- The third line contains an integer k, the maximum number of routers available.

Output

Print the **minimum integer radius** required to cover all houses using at most k routers.

Test Case 1

Input

5
1 2 4 8 9
3

Output

1

Explanation

We can install routers to cover:

Router 1 → houses at 1, 2
Router 2 → house at 4
Router 3 → houses at 8, 9

Therefore, radius 1 is sufficient.

Test Case 2

Input

6
1 2 3 10 11 12
2

Output:1

1

Explanation

We can cover:

Router 1 → 1, 2, 3
Router 2 → 10, 11, 12

With radius 2, all houses are covered using 2 routers.

Test Case 3

Input

5
1 5 10 15 20
2

Output

5

Explanation

One optimal arrangement is:

Router 1 → 1, 5, 10
Router 2 → 15, 20

A radius of 5 is sufficient.

A radius smaller than 5 cannot cover all houses with only 2 routers.

Test Case 4

Input

7
2 3 4 8 9 10 20
3

Output

1

Explanation

Using radius 1:

Router 1 → 2, 3, 4
Router 2 → 8, 9, 10
Router 3 → 20

So all houses can be covered using 3 routers.

Test Case 5

Input

8
1 2 3 4 20 21 22 23
2

Output

2

Explanation

We can cover:

Router 1 → 1, 2, 3, 4
Router 2 → 20, 21, 22, 23

A radius of 2 is sufficient.

Constraints

1 ≤ n ≤ 10^5

1 ≤ k ≤ n
0 ≤ house position ≤ 10^9

Solution:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

using ll = long long;

// Check if all houses can be covered using at most k routers
// when each router has radius R.
bool canCover(vector<ll>& houses, int k, ll R) {

    int n = houses.size();
    int routers = 0;
    int i = 0;

    while (i < n) {

        // We are going to place a router to cover houses[i].
        routers++;

        // Put the router as far right as possible
        // while still covering houses[i].
        ll routerPosition = houses[i] + R;

        // This router can cover houses up to:
        // routerPosition + R
        ll coverUntil = routerPosition + R;

        // Skip all houses covered by this router
        while (i < n && houses[i] <= coverUntil) {
            i++;
        }

        // If we need more than k routers, R is not possible.
        if (routers > k) {
            return false;
        }
    }

    return true;
```

```
}

int main() {

    int n;
    cin >> n;

    vector<ll> houses(n);

    for (int i = 0; i < n; i++) {
        cin >> houses[i];
    }

    int k;
    cin >> k;

    // Houses must be sorted for the greedy approach.
    sort(houses.begin(), houses.end());

    // Minimum possible radius
    ll low = 0;

    // Maximum possible radius.
    // One router placed appropriately can cover everything
    // if R >= (max - min) / 2.
    //
    // Using max-min as a safe upper bound is simpler.
    ll high = houses[n - 1] - houses[0];

    ll answer = high;

    while (low <= high) {

        ll mid = low + (high - low) / 2;

        if (canCover(houses, k, mid)) {

            // mid works.
            // Try a smaller radius.
            answer = mid;
            high = mid - 1;
        }
        else {
```

```
            // mid does not work.
            // We need a larger radius.
            low = mid + 1;
        }
    }

    cout << answer << endl;

    return 0;
}
```